

homework 5

1. פרמטרים של ברירת מחדל (Default Parameters) והשימוש בהם

מהם פרמטרים של ברירת מחדל?

פרמטרים של ברירת מחדל הם פרמטרים המוגדרים בחתימת הפונקציה יחד עם ערך התחלתי מוגדר מראש. אם בעת הקריאה לפונקציה לא מועבר ערך עבור פרמטר זה, הפונקציה תשתמש אוטומטית בערך ברירת המחדל שהוגדר לה. תכונה זו מאפשרת גמישות רבה ומצמצמת את הצורך בכתיבת מספר גרסאות של אותה פונקציה.

האם חובה לשלוח את כל הפרמטרים בקריאה לפונקציה?

לא תמיד. החובה תלויה בסוג הפרמטר:

- פרמטרי חובה (Positional Arguments): פרמטרים שאין להם ערך ברירת מחדל הם פרמטרי חובה. חובה לספק להם ערך בעת הקריאה לפונקציה, אחרת תתקבל שגיאה.
- פרמטרים אופציונליים (Default Arguments): פרמטרים שיש להם ערך ברירת מחדל אינם דורשים העברת ערך בקריאה לפונקציה. אם נשלח ערך – הוא ידרוס את ברירת המחדל; אם לא נשלח ערך – המערכת תשתמש בערך ברירת המחדל.

האם סדר הפרמטרים משנה בעת הקריאה לפונקציה?

התשובה תלויה באופן שבו אנו מעבירים את הערכים (בדוגמאות נתייחס לפייתון, המייצגת את ההתנהגות ברוב השפות המודרניות):

- העברה לפי מיקום (Positional): כאשר מעבירים ערכים ללא ציון שם הפרמטר, הסדר משנה בהחלט. הערך הראשון ישויך לפרמטר הראשון, השני לשני, וכן הלאה.
- העברה לפי שם (Keyword Arguments): שפות רבות מאפשרות להעביר ערכים תוך ציון מפורש של שם הפרמטר (למשל "name=Alice"). במקרה כזה, הסדר אינו משנה, כיוון שהמפרש יודע בדיוק לאיזה פרמטר לשייך כל ערך.

האם סדר הפרמטרים משנה בעת הגדרת הפונקציה?

כן, לסדר יש חשיבות מכרעת. כלל ברזל בתכנות הוא שבעת חתימת (הגדרת) הפונקציה, כל פרמטרי החובה חייבים להופיע לפני הפרמטרים של ברירת המחדל. אם נגדיר פרמטר חובה אחרי פרמטר ברירת מחדל, המהדר (או המפרש) יחזיר שגיאת תחביר (Syntax Error), שכן הוא לא יוכל להבין אילו ערכים בקריאה שייכים לאילו פרמטרים.

2. העמסת פונקציות (Function Overloading)

מהי העמסת פונקציות?

העמסת פונקציות היא תכונה מרכזית בשפות תכנות רבות (כגון C++, Java ו-C#) המאפשרת להגדיר מספר פונקציות שונות החולקות את אותו השם בדיוק, באותה תוכנית או באותה מחלקה. המטרה של העמסת פונקציות היא לשפר את קריאות הקוד, להפוך אותו לאינטואיטיבי יותר, ולאפשר לאותה פעולה לוגית להתבצע על סוגים שונים או כמויות שונות של נתונים, מבלי שנצטרך להמציא שם חדש לכל וריאציה (למשל, במקום לקרוא לפונקציות בשמות מסורבלים כמו `addIntegers` ו-`addDoubles`, נקרא לכולן פשוט `add`).

האם מותר לשתי פונקציות להיקרא באותו שם?

התשובה היא כן, אך בתנאי אחד קריטי: "חתימת הפונקציה" (Function Signature) שלהן חייבת להיות שונה. כלומר, לא ניתן ליצור שתי פונקציות עם אותו שם ואותם פרמטרים בדיוק. כדי שהמחדר (Compiler) ידע להבדיל ביניהן ולבחור את הפונקציה הנכונה בזמן קריאה, הפונקציות חייבות להיות שונות בלפחות אחד מהמרכיבים הבאים ברשימת הפרמטרים שלהן:

- **מספר הפרמטרים:** לפונקציה אחת יש שני פרמטרים ולשנייה יש שלושה.
 - **סוגי הפרמטרים (Data Types):** פונקציה אחת מקבלת מספרים שלמים (`int`) והשנייה מקבלת מספרים עשרוניים (`double`).
 - **סדר הפרמטרים:** פונקציה אחת מקבלת (`int, double`) והשנייה מקבלת (`double, int`).
- הערה חשובה: סוג הערך המוחזר מהפונקציה (`Return Type`) בלבד אינו מספיק כדי להבדיל בין פונקציות. אם לשתי פונקציות יש את אותו שם ואותם פרמטרים בדיוק, אך הן מחזירות סוג נתונים שונה, תתקבל שגיאת קומפילציה.

3. החזרת ערכים מרובים מפונקציה

כמה ערכים מותר להחזיר מפונקציה?

באופן תיאורטי ורשמי, פונקציה מחזירה תמיד ערך אחד ויחיד. עם זאת, בשפות תכנות מודרניות רבות (כמו Python), הערך הבודד הזה יכול להיות מבנה נתונים (אוסף) שמכיל בתוכו מספר פריטים (כמו למשל `Tuple` - רשומה, רשימה או מילון). לכן, בפועל, ניתן להחזיר דרך הפונקציה כל כמות של ערכים שרוצים. כאשר אנו כותבים פקודת `return` עם מספר ערכים מופרדים בפסיקים, המערכת אוטומטית "אורזת" אותם למבנה נתונים אחד (לרוב `Tuple`) ומחזירה אותו.

כיצד משייכים (מציבים) מספר ערכים שחזרו מפונקציה?

הדרך האלגנטית והנפוצה ביותר (במיוחד בפייתון) נקראת "פריקה" (`Unpacking`). בעת הקריאה לפונקציה, אנו כותבים בצד השמאלי של פעולת ההשמה (הסימן `=`) מספר משתנים מופרדים בפסיקים. הערכים שחוזרים מהפונקציה "יפורקו" ויוצבו בתוך המשתנים הללו, בהתאמה מדויקת לסדר שבו הם הוחזרו.

האם חובה להתייחס לכל הערכים שמוחזרים?

לא, ממש לא חובה. יש לנו גמישות מלאה לגבי אופן ההתייחסות לערכים החוזרים:

1. **השמה למשתנה בודד:** אפשר פשוט להציב את הקריאה לפונקציה לתוך משתנה אחד. במקרה כזה, המשתנה יכול את מבנה הנתונים השלם (למשל, את ה-Tuple כולו) המאגד את כל הערכים יחד.
2. **התעלמות מערכים ספציפיים בעת פריקה:** כאשר אנו מבצעים "פריקה" למספר משתנים, חובה לספק משתנה עבור כל ערך שחוזר. עם זאת, אם אנו צריכים רק חלק מהערכים ולא אכפת לנו מהשאר, המוסכמה המקובלת היא להשתמש בתו קו תחתון (_) כשם של משתנה "פח אשפה" – זה מאותת לקורא הקוד (וגם למערכת בחלק מהמקרים) שאנו מודעים לכך שיש שם ערך, אך אנו בוחרים להתעלם ממנו ולא להשתמש בו בהמשך.

4. "אזרח מדרגה ראשונה" (First-Class Citizen) בתכנות

מהו "אזרח מדרגה ראשונה" בתכנות?

"אזרח מדרגה ראשונה" (או First-Class Object/Citizen) הוא מושג במדעי המחשב המתאר ישות בשפת התכנות שתומכת בכל הפעולות הבסיסיות שזמינות לישויות אחרות (כמו מספרים, מחרוזות או מערכים). בשפות תכנות מודרניות רבות (כגון Python ו-JavaScript), **פונקציות** נחשבות לאזרחיות מדרגה ראשונה (First-Class Functions).

כדי שישות, כמו פונקציה, תיחשב לאזרח מדרגה ראשונה, היא חייבת לעמוד בשלושה תנאים מרכזיים:

1. ניתן לשמור (להציב) אותה בתוך משתנה או במבנה נתונים.
2. ניתן להעביר אותה כארגומנט (פרמטר) לפונקציה אחרת.
3. ניתן להחזיר אותה כערך (Return value) מתוך פונקציה.

האם ניתן להציב (להקצות) פונקציה לתוך משתנה?

כן, בהחלט. בשפות שבהן פונקציות הן אזרחיות מדרגה ראשונה, הפונקציה היא אובייקט לכל דבר. לכן, ניתן להגדיר משתנה ולהציב בתוכו את הפונקציה עצמה. חשוב לשים לב שמבצעים את ההשמה ללא סוגריים (הוספת סוגריים תפעיל את הפונקציה ותציב את התוצאה שלה במשתנה, בעוד שללא סוגריים אנו מציבים את הפונקציה עצמה). לאחר ההצבה, המשתנה משמש למעשה כשם חלופי (Alias) לפונקציה וניתן להפעיל אותה דרכו.

האם ניתן להחזיר פונקציה כערך מתוך פונקציה אחרת?

כן. פונקציה יכולה להגדיר בתוכה פונקציה פנימית חדשה (או להשתמש בפונקציה קיימת) ולהחזיר אותה החוצה באמצעות פקודת ה-`return`, בדיוק כפי שהייתה מחזירה מספר שלם או רשימה. פונקציות שמסוגלות לקבל פונקציות אחרות כפרמטרים או להחזיר פונקציות אחרות כתוצאה נקראות "**פונקציות מסדר גבוה**" (Higher-Order Functions).

5. פונקציית למבדה (Lambda Function) ופונקציות אנונימיות

מהי פונקציית למבדה?

פונקציית למבדה (בשפות כמו Python) היא פונקציה קטנה, תמציתית הנכתבת בשורה אחת, ומוגדרת באמצעות מילת המפתח `lambda` במקום `def`.

מה ההבדל בינה לבין פונקציה רגילה?

- **תחביר:** פונקציה רגילה מוגדרת עם `def` ויכולה להכיל בלוק קוד שלם של מספר שורות. למבדה מוגדרת עם `lambda` ומוגבלת לביטוי (Expression) בודד בשורה אחת.
- **החזרת ערך:** בפונקציה רגילה יש להשתמש במפורש במילה `return`. בפונקציית למבדה, התוצאה של הביטוי מוחזרת אוטומטית (Implicit return).
- **שם הפונקציה:** פונקציות רגילות חייבות שם. פונקציות למבדה הן "אנונימיות" כברירת מחדל.

מה היתרון בשימוש בה?

היתרון המרכזי הוא קוד קצר, נקי וקריא יותר כאשר אנו זקוקים לפונקציה פשוטה מאוד לשימוש חד-פעמי. במקום להגדיר פונקציה שלמה עם שם עבור פעולה טריוויאלית, כותבים למבדה במקום. זה שימושי במיוחד בעבודה עם פונקציות מסדר גבוה כמו `filter`, `map` או `sort` (שמקבלות פונקציות כפרמטר).

מה משמעות המושג "פונקציה אנונימית" (Anonymous Function)?

הכוונה היא לפונקציה שנוצרת ללא מזהה (שם) שקשור אליה. בניגוד ל-`def` שיוצר אובייקט של פונקציה וקושר אותו מיד לשם שבחרנו, `lambda` פשוט יוצרת את האובייקט.

איך מקצים לה ערך (משייכים אותה)?

ניתן להקצות את פונקציית הלמבדה למשתנה, וכך לתת לה שם ולזהות אותה:

```
# למשתנה והשמה למבדה פונקציית הגדרת
multiply = lambda x, y: x * y

# לפונקציה קריאה
result = multiply(5, 3)
print(result) # יודפס: 15
```

6. דקורטור (Decorator)

מהו דקורטור ולמה אנחנו משתמשים בו?

דקורטור הוא תבנית עיצוב (Design Pattern) המאפשרת להוסיף התנהגות או פונקציונליות לפונקציה קיימת באופן דינמי, מבלי לשנות את קוד המקור שלה. בפייתון, דקורטור הוא פשוט פונקציה שמקבלת פונקציה אחרת כארגומנט, ו"עוטפת" אותה כדי להרחיב את הפעולה שלה לפני או אחרי שהיא רצה.

אנו משתמשים בהם כדי לשמור על קוד נקי ולמנוע כפילויות (DRY) – למשל עבור לוגיקה שחוזרת על עצמה כמו בדיקת הרשאות (Authentication), מדידת זמני ריצה או רישום לוגים (Logging).

האם ניתן לעשות זאת עם פונקציה רגילה אחרת?

כן, **בהחלט**. הסינטקס המוכר של דקורטור עם סמל ה-@ (למשל @my_decorator) הוא בסך הכל "סוכר תחבירי" (Syntactic Sugar). מתחת למכסה המנוע, הדקורטור עצמו הוא פונקציה רגילה לחלוטין (פונקציה מסדר גבוה).

הקוד decorator@ מעל הפונקציה func פשוט שקול להרצת השורה הרגילה: func = decorator(func)

```
# כדקורטור המשמשת רגילה פונקציה
def my_logger(func):
    def wrapper():
        print("Something is happening before the function is called.")
        func()
        print("Something is happening after the function is called.")
    return wrapper

@my_logger
def say_whee():
    print("Whee!")

say_whee()
```

codesnap.dev

7. ההבדל בין map ל-filter

מה ההבדל בין map ל-filter?

- map : מאוסף איברים. היא מפעילה את הפונקציה על כל אחד מהאיברים באוסף ומחזירה : אוסף חדש עם התוצאות שעברו טרנספורמציה (שינוי).

- ואוסף איברים. היא מסננת את האוסף ומחזירה (True/False שמחזירה) מקבלת פונקציית תנאי : `filter`. True . רק את האיברים שעבורם פונקציית התנאי החזירה

האם אחת יכולה להוות תחליף לשנייה?

לא. למרות ששתיהן פונקציות מסדר גבוה הפועלות על רשימות, מטרתן שונה בתכלית. `map` משנה ערכים אך לא משנה את כמותם, בעוד ש- `filter` לא משנה את הערכים עצמם אלא רק מקטינה את כמותם (מסננת).

מהו אורך הרשימה המוחזרת בכל אחת מהן?

- אורך הרשימה המוחזרת תמיד שווה לאורך הרשימה המקורית שנשלחה אליה : `map`.
- אורך הרשימה המוחזרת יהיה קטן או שווה לאורך הרשימה המקורית (בהתאם לכמות האיברים : `filter`). (שעמדו בתנאי).

```
nums = [1, 2, 3, 4]

# map: (איברים 4 : זהו אורך) 2 פי איבר כל מכפיל
mapped_nums = list(map(lambda x: x * 2, nums)) # [2, 4, 6, 8]

# filter: (איברים 2 : יותר קטן אורך) זוגיים מספרים רק משאיר
filtered_nums = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
```

codesnap.dev

8. כיצד מועברים ערכים בפיתון? (...Pass by)

בפיתון מנגנון העברת הפרמטרים לפונקציות אינו "העברה לפי ערך" (Pass by Value) קלאסי ואינו "העברה לפי הפניה" (Pass by Reference) קלאסי (כמו ב-C++). המנגנון בפיתון נקרא "העברה לפי הפניה לאובייקט" (Pass by Object Reference) או "העברה על ידי השמה" (Pass by Assignment).

****איך זה עובד****

כאשר מעבירים משתנה לפונקציה, מועבר "רפרנס" (הפניה למיקום בזיכרון) של האובייקט, אך הפניה זו מועברת כערך אל המשתנה המקומי של הפונקציה. ההתנהגות תלויה בסוג האובייקט:

1. **אובייקטים שניתנים לשינוי (Mutable - כמו List, Dictionary):** אם נשנה את תוכן האובייקט בתוך הפונקציה (למשל, נוסיף איבר לרשימה), השינוי יורגש גם מחוץ לפונקציה (כי שניהם מצביעים לאותו אובייקט שניתן לעריכה).
2. **אובייקטים שאינם ניתנים לשינוי (Immutable - כמו Int, String, Tuple):** מכיוון שלא ניתן לשנות אותם, כל ניסיון לבצע השמה חדשה למשתנה בתוך הפונקציה פשוט יצור אובייקט חדש במרחב הזיכרון המקומי,

9. משתנים גלובליים מול מקומיים (Global vs Local)

משתנה גלובלי (Global Variable):

- היכן נוצר? מוגדר מחוץ לכל הפונקציות, לרוב ברמה העליונה של הקובץ (המודול).
- היכן ניתן להשתמש בו? ניתן לקרוא אותו מכל מקום בקוד, גם מחוץ וגם בתוך כל פונקציה. (הערה: כדי לשנות את ערכו מתוך פונקציה, יש להשתמש במפורש במילת המפתח `global`).

משתנה מקומי (Local Variable):

- היכן נוצר? מוגדר בתוך בלוק של פונקציה ספציפית. נוצר בזיכרון ברגע שהפונקציה מופעלת, ונמחק מהזיכרון כאשר הפונקציה מסיימת לרוץ.
 - היכן ניתן להשתמש בו? אך ורק בתוך תחום (Scope) הפונקציה שבה הוגדר. ניסיון לגשת אליו מחוץ לפונקציה יוביל לשגיאה (`NameError`).
-

10. פרמטר פורמלי לעומת פרמטר אקטואלי

- פרמטר פורמלי (Formal Parameter): זהו "שם המשתנה" (מקום שמור) המופיע בחתימת הפונקציה (בהגדרת הפונקציה). תפקידו לייצג את הנתון שהפונקציה מצפה לקבל כשהיא מוגדרת.
- פרמטר אקטואלי / ארגומנט (Actual Parameter / Argument): זהו הערך (או המשתנה) האמיתי, הפיזי, שנשלח בפועל לפונקציה בזמן הקריאה אליה (Function Call).

למה אנו צריכים את שניהם?

אנו חייבים את שניהם בגלל עקרון ההפשטה והשימוש החוזר (Abstraction and Reusability). הפרמטרים הפורמליים מאפשרים לנו לכתוב את הלוגיקה של הפונקציה פעם אחת, תוך שימוש בשמות גנריים, מבלי לדעת מראש אילו נתונים יוזנו אליה. הפרמטרים האקטואליים מאפשרים לנו להפעיל את אותה לוגיקה שנכתבה, שוב ושוב, על נתונים שונים וספציפיים בכל פעם במהלך ריצת התוכנית.